Aerospace Information Technology
Online FloatSat 2021

Floating Satellite System (FloatSat)

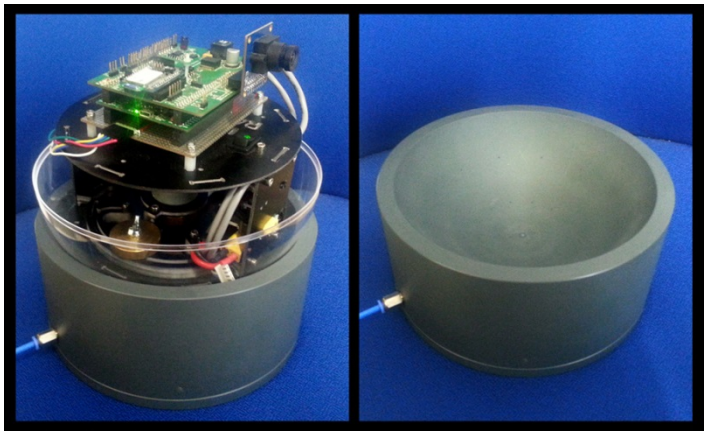
Technical Presentation (Actuators & Sensors)

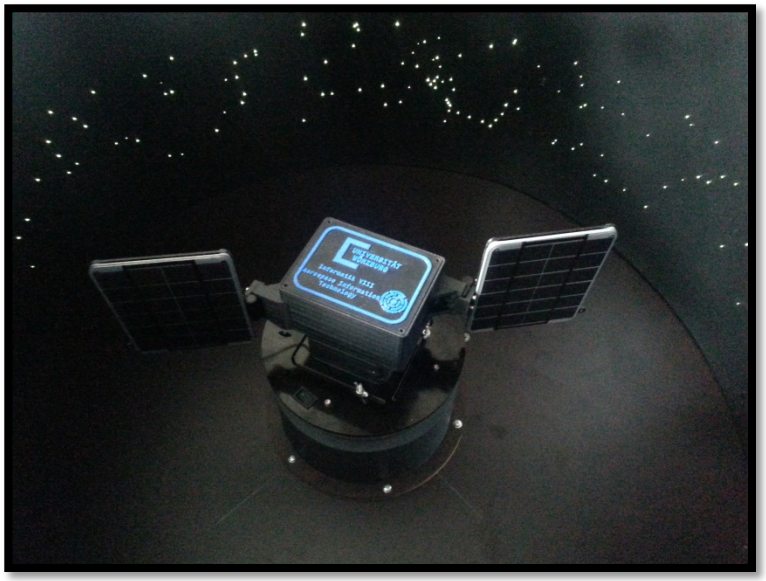
M.Sc. Eng. Atheel Redah

The Floating Satellite (FloatSat) System:

● The Chair of Aerospace Information Technology at the University of Würzburg (Germany) has developed a Floating Satellite (FloatSat) system in order to help students understand and get familiar with basic satellite subsystems and also to develop and test different attitude control algorithms and strategies for small satellites in an almost frictionless environment similar to that in space.

● The FloatSat system is consists mainly from a mechanical structure that contains the basic satellite subsystems. This structure is placed into a hemisphere shell that it is floating inside a Spherical Air Bearing unit. The air bearing unit requires compressed air input may vary depending on the mass of the floating unit. In order to monitor and command the FloatSat, a ground station computer could be used to communicate via the Bluetooth module available in the satellite through commanding and telemetry display tool.

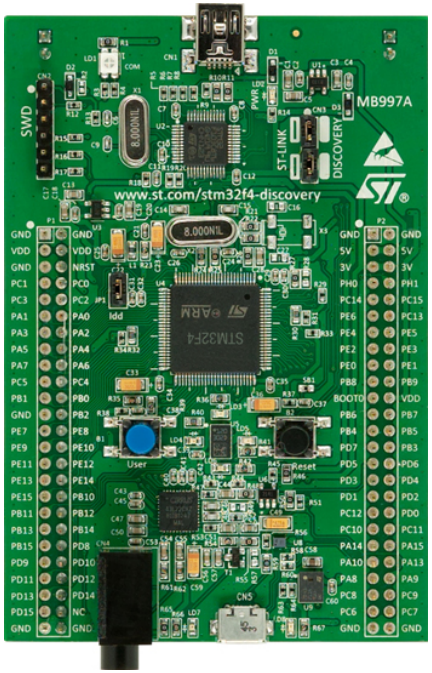




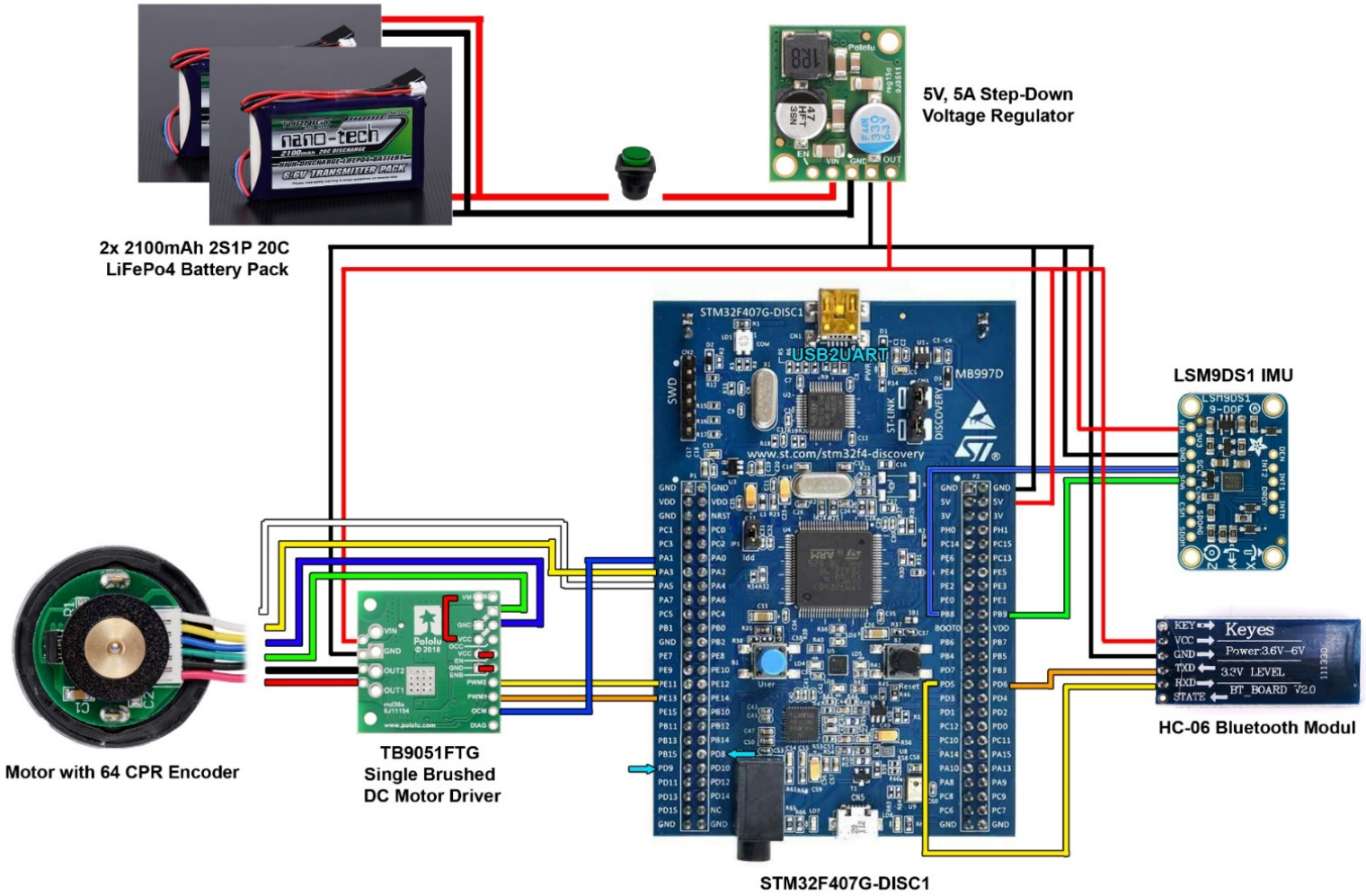
Floating Satellite System (FloatSat)

Embedded System

The STM32F4DISCOVERY Development Board:



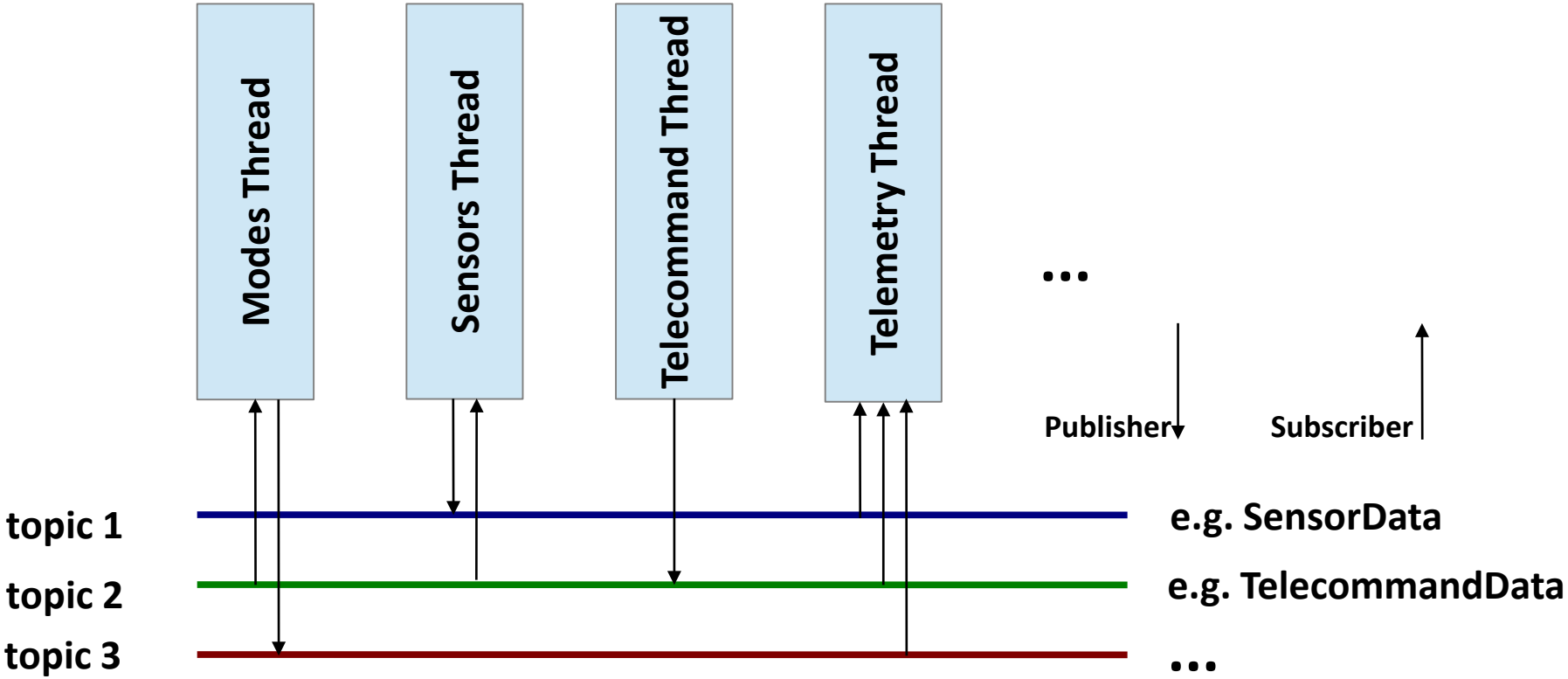
- **STMicroelectronics** – www.st.com.
- **Programmer:** On-board ST-LINK/V2 in-circuit debugger and programmer.
- **Board power supply**: Through USB bus or from an external 5 V supply voltage.
- **Core:** ARM 32-bit Cortex™-M4 CPU with FPU, ART Accelerator™ (Adaptive real-time accelerator) allowing 0-wait state execution from Flash memory, frequency up to 168 MHz, MPU (Memory Protection Unit), 210 DMIPS/1.25 DMIPS/MHz (Dhrystone 2.1), and DSP instructions.
- **Memories:** 1 MB of Flash memory (Application), 192+4 KB of SRAM including 64KB of CCM (Core Coupled Memory) data RAM (Working).
- **3×12-bit A/D converters:** up to 24 channels with 2.4 MSPS and 7.2 MSPS in triple interleaved mode.
- **2×12-bit D/A converters**.
- **General-purpose DMA:** 16-stream DMA controller with FIFOs and burst support.
- **17 x Timers:** up to 12x16-bit and 2x32-bit timers up to 168 MHz, each with up to 4 IC/OC/PWM or pulse counter and quadrature (incremental) encoder input.
- **82 x GPIO pins:** with interrupt capability.
- **15 communication interfaces:** 3×I2C interfaces (SMBus/PMBus), 4xUSARTs / 2xUARTs (10.5 Mbit/s, ISO 7816 interface, LIN, IrDA, modem control), 3xSPIs (42 Mbits/s), 2×CAN interfaces (2.0B Active), SDIO interface.
- **Advanced connectivity:** USB 2.0 full-speed device/host/OTG controller with on-chip PHY, USB 2.0 high-speed/full-speed device/host/OTG controller with dedicated DMA, on-chip full-speed PHY and ULPI–10/100 Ethernet MAC with dedicated DMA which supports IEEE 1588v2 hardware, MII/RMII.
- **8- to 14-bit parallel DCMI** (Digital Camera Interface) up to 54 Mbytes/s, and parallel **LCD interface** (8080/6800 modes).
- True **RNG** (Random Number Generator) and **CRC** (Cyclic Redundancy Check) calculation unit.



FloatSat Wiring Diagram

ADC		TIM		I2C		UART		GPIO
ADC123_IN1 [ADC_CH_001]	PA1 [Motor Current sense]	TIM1_CH2 [PWM_IDX01]	PE11 [PWM2]	I2C1_SCL [I2C_IDX1]	PB8 [IMU_SCL]	USART2_TX [UART_IDX2]	PD5 [BT2UART]	PD12 [LED Green]
		TIM1_CH3 [PWM_IDX02]	PE13 [PWM1]	I2C1_SDA [I2C_IDX1]	PB9 [IMU_SDA]	USART2_RX [UART_IDX2]	PD6 [BT2UART]	PD13 [LED Orange]
								PD14 [LED Red]
		TIM2_CH4 [Input Capture mode]	PA3 [Encoder A]			USART3_TX [UART_IDX3]	PD8 [USB2UART]	PD15 [LED Blue]
						USART3_RX [UART_IDX3]	PD9 [USB2UART]	
								PA5 [Encoder B]

FloatSat:


Modes Thread
Sensors Thread
Telecommand Thread
Telemetry Thread
...
Publisher
Subscriber
topic 1
topic 2
topic 3
e.g. SensorData
e.g. TelecommandData
...

Floating Satellite System (FloatSat)

Modules & Sensors

Inertial Measurement Unit (IMU):

● Inertial Measurement Unit (IMU), is an electronic device that measures and reports on an object's velocity, orientation, and gravitational forces, using a combination of accelerometers, gyroscopes, and magnetometers.

● The LSM9DS1 is an inertial measurement unit (IMU) that packs an 3-axis Gyroscope with 3-axis Accelerometer and 3-axis Magnetometer:

* SPI / I2C serial interfaces.

* 16-bit data output per axis.

* Sensitivity range (configurable):
 - Gyro: ±245, ±500, or ±2000 dps angular rate full scale.
 - Accelerometer: ±2, ±4, ±8 or ±16 g linear acceleration full scale.
 - Magnetometer: ±4, ±8, ±12, or ±16 gauss magnetic full scale.

* Embedded temperature sensor.

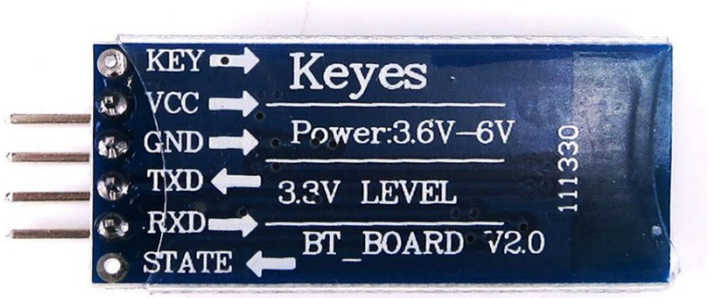
● Orientation Angles:
- Three attitude angles are referenced to the local horizontal plane which is perpendicular to earth's gravity:
* Yaw is defined as the angle between the sensor X-axis and the magnetic north on the horizontal plane.
* Pitch is defined as the angle between the sensor X-axis and the horizontal plane.
* Roll is defined as the angle between the sensor Y-axis and the horizontal plane.

Wireless Modules:

- **HC-06 Wireless Bluetooth Modul:**

- Class 2 module with PCB antenna with
- Transmit power: 4dBm
- Support Serial Port Profile (SPP) in Slave Mode
- Configurable with AT commands
- Bluetooth standard Ver. 2.0 + EDR compliant
- Power Supply: +3.3VDC 50mA

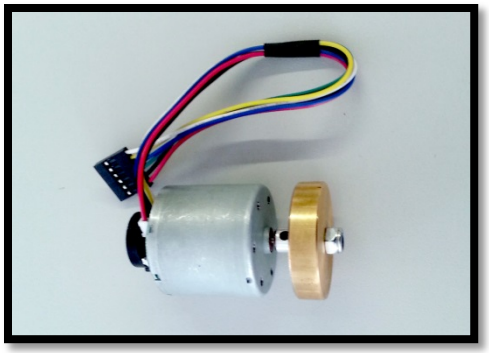
- The default setting for serial port [9600, N, 8,1], password: 1234.
- The actual setting for serial port [115200, N, 8,1], password: 1234.



Floating Satellite System (FloatSat)

Actuators

Reaction Wheel:



- A reaction wheel or a momentum wheel in a spacecraft is an electric motor attached to a flywheel.
- When the wheel rotation speed is changed, a torque is generated:

$$\tau = J_w \frac{d\omega_w}{dt}$$

Where J_w is the wheel moment of inertia and ω_w is the wheel rotation speed.

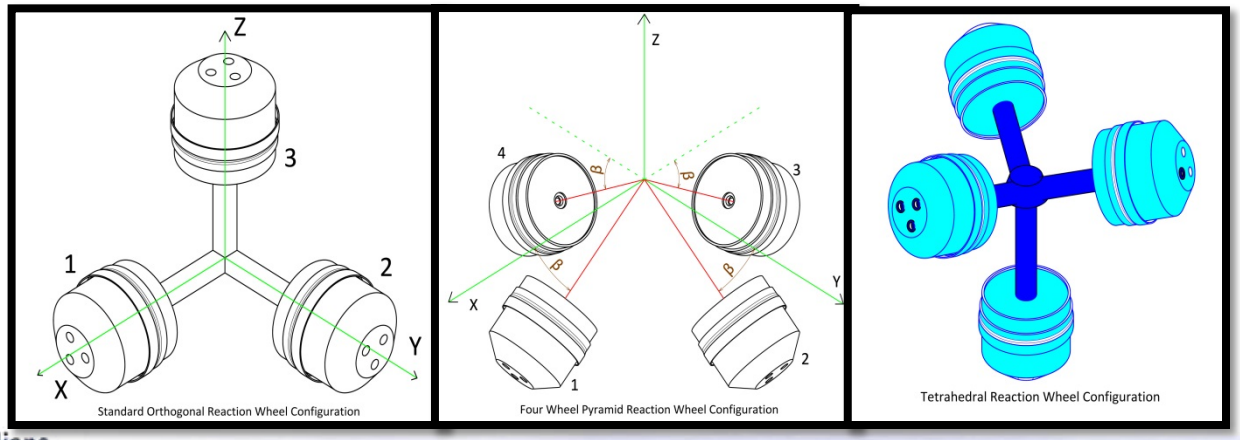
- According to conservation of angular momentum law, the generated torque will cause the spacecraft to counter rotate proportionally around its center of mass in the same plane of rotation:

$$\tau_s = -J_w \frac{d\omega_w}{dt} \rightarrow J_s \frac{d\omega_s}{dt} = -J_w \frac{d\omega_w}{dt}$$

$$\omega_s = -\frac{J_w}{J_s} * \omega_w \; and \; \theta_s = -\frac{J_w}{J_s} * \int_0^t \omega_w . d\tau$$

Where J_s is the spacecraft moment of inertia, ω_s is the spacecraft rotation speed and θ_s is the spacecraft angular position.

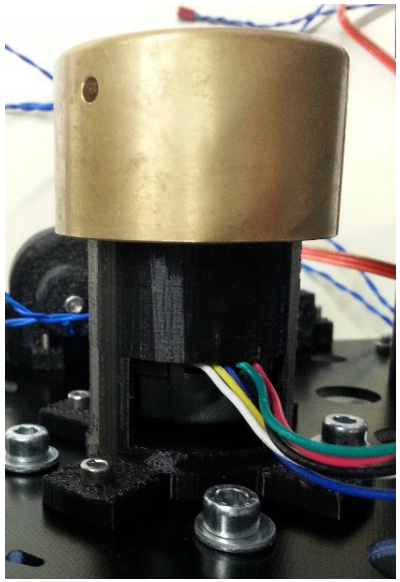
- For precision three axis attitude control of a spacecraft, at least three reaction wheels are needed:

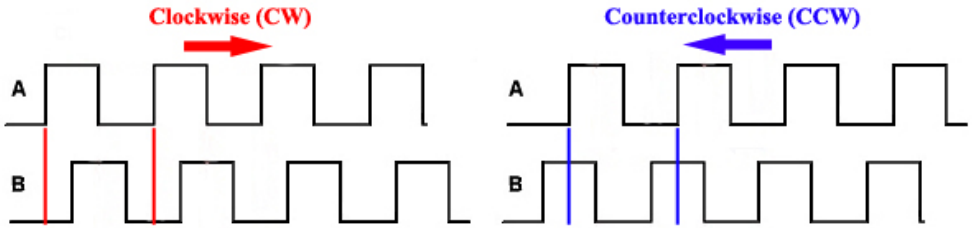


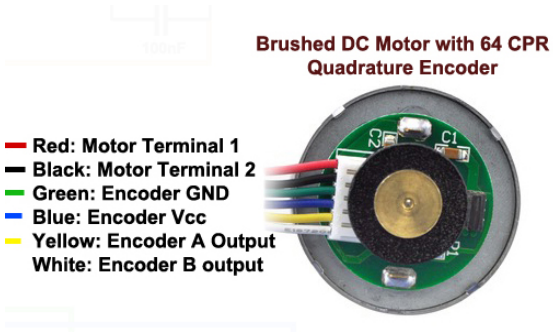
Actuator & Power Driver:

Motorized Reaction Wheel:

- **Brushed Motor with 64 CPR Encoder**
- **1-12V** Operating voltage.
- Key specs at **12 V**: **11,000 RPM** and **300 mA** free-run, **0.3 kg-cm** and **5 A stall**.
- Integrated **64 CPR** (counts per revolution) quadrature **Hall effect encoder** on **two channels** (each channel has 32 CPR).
- **Reaction Wheel**: **298 g @ 1.175*10-4 Kgm2**
- **The quadrature encoder has two square wave outputs (A and B) which are 90 degrees out of phase with each other**



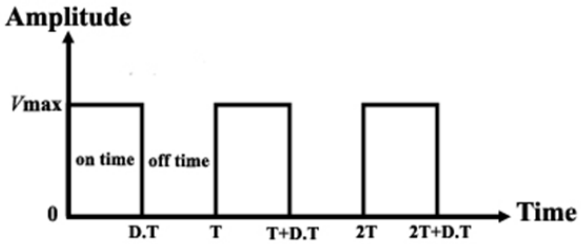




Pulse Width Modulation (PWM):

● Pulse Width Modulation (PWM) is a powerful technique for controlling analog systems that expects an analog signal of varying amplitude with microcontroller's digital outputs:

* The voltage or current source is supplied to the analog load by means of repeating series of on and off pulses.
* The on-time is the time during which the DC supply is applied to the load.
* The off-time is the times during which that supply is switched off.
* The term duty cycle D describes the proportion of on-time to the regular interval or period of time T for the PWM signal. Duty cycle is expressed in percent, where 100% being fully on and 0% being fully off.
* PWM controlled drivers regulate the effective applied voltage by changing the duty ratio of on-time to off-time at a given frequency.



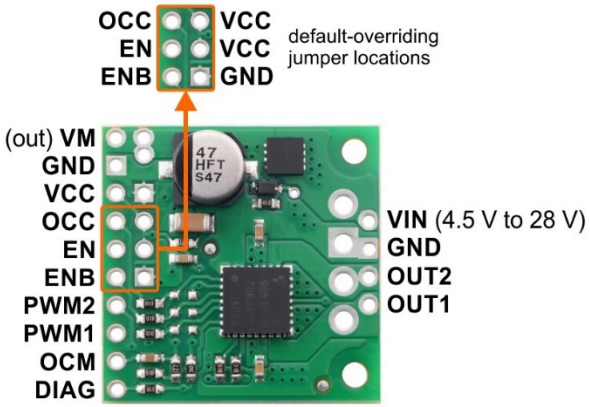
* If the pulse waveform has a low value equal to zero, a high value equal to $V_{\max}$ and a duty cycle D, then the average value of the waveform V_{ave} can be given by following equation :

$$V_{ave} = \frac{1}{\mathrm{T}} \int_0^{\mathrm{DT}} V_{\max} \cdot dt = \mathrm{D} V_{\max}$$

Actuator & Power Driver:

Power Driver:

- **Pololu TB9051FTG Single Brushed DC Motor Driver(H-Bridge)**
- Motor supply voltage: 4.5 V to 28 V
- Output current: up to 2.6 A continuous (5 A peak)
- **Under-voltage lockout and protection against over-current/short-circuit and over-temperature**
- **Carrier board adds reverse-voltage protection**
- Peak output current per channel: **5 A**
- Current sense: **0.5 V/A**
- Maximum PWM frequency: **20 kHz**
- **Two PWM signals are needed for drive-brake operation (slow decay).**



TB9051FTG simplified truth table (PWM1 + PWM2)						
Inputs				Outputs		Operation
EN	ENB	PWM1	PWM2	OUT1	OUT2	
1	0	PWM	0	PWM (H/L)	L	forward/brake at speed *PWM %*
		0	PWM	L	PWM (H/L)	reverse/brake at speed *PWM %*
		0	0	L	L	brake low (outputs shorted to ground)
		1	1	L	L	
0	X	X	X	Z	Z	coast (outputs floating/disconnected)
X	1	X	X	Z	Z	

RODOS HAL (PWM):

```
class HW_HAL_PWM; /* hal_pwm.h */
class HAL_PWM : public GenericIOInterface { ///< one object per pwm
HW_HAL_PWM* context;
public:
HAL_PWM(PWM_IDX idx);

/* Initialize PWM without starting it -> start with "write(N)"
 * To get exact PWM frequency you must choose frequency and increments so that timerclock/frequency/increments = integer (no fraction)
     * e.g. STM32F4 Timer 1: 168Mhz/1000/100 = 1680 :), but 168Mhz/1000/128 = 1315.5 -> PWM freq = 168MHz/1315/128 = 998.1Hz :(
     *
     * STM32F4  : - Tim1 (16 bit,168MHz), Tim2 (32 bit,84MHz), Tim3 (16 bit,84MHz), Tim4 (16 bit,84MHz)
     *            - PWMs of the same timer can only run with same settings
 * @param int frequency - frequency of PWM signal
 * @param int increments - one PWM period is divided in N increments */
int init(int frequency = 1000, int increments = 100);
int config(PWM_PARAMETER_TYPE type, int paramVal = 0);
void reset();
bool isWriteFinished() { return true; }

 /*** Extra only for PWM **/
 * Set pulse width of PWM signal
 * - write(increments/2) -> duty cycle 50%
 * - write(0) -> stop PWM-output
 * @param int pulseWidthInIncs */
int write(unsigned int pulseWidthInIncs);// 0 ... +increments
};
```

```
enum PWM_PARAMETER_TYPE {
PWM_PARAMETER_FREQUENCY,
PWM_PARAMETER_INCREMENTS};
```

```
enum PWM_IDX {      // STM32F4
PWM_IDX00 = 0,  // Timer 1 CH1 - PE9
PWM_IDX01,      // Timer 1 CH2 - PE11
PWM_IDX02,      // Timer 1 CH3 - PE13
PWM_IDX03,      // Timer 1 CH4 - PE14
PWM_IDX04,      // Timer 2 CH1 - PA0
PWM_IDX05,      // Timer 2 CH2 - PA1
PWM_IDX06,      // Timer 2 CH3 - PA2
PWM_IDX07,      // Timer 2 CH4 - PA3
PWM_IDX08,      // Timer 3 CH1 - PA6
PWM_IDX09,      // Timer 3 CH2 - PC7
PWM_IDX10,      // Timer 3 CH3 - PB0
PWM_IDX11,      // Timer 3 CH4 - PB1
PWM_IDX12,      // Timer 4 CH1 - PD12
PWM_IDX13,      // Timer 4 CH2 - PD13
PWM_IDX14,      // Timer 4 CH3 - PD14
PWM_IDX15       // Timer 4 CH4 - PD15
};
```

Example:

```
HAL_PWM pwm1(PWM_IDX02); /* declare HAL_PWM for PWM_IDX02 = TIM1-CH3 (PE13), please refer to hal_pwm.h for correct PWM mapping*/
HAL_PWM pwm2(PWM_IDX01); /* declare HAL_PWM for PWM_IDX01 = TIM1-CH2 (PE11), please refer to hal_pwm.h for correct PWM mapping*/

pwm1.init(4000, 1000);        /* initialization of the HAL object should be called one time only in the project*/
pwm2.init(4000, 1000);        /* initialization of the HAL object should be called one time only in the project*/

pwm1.write(250);              /* Set the Duty Cycle for pwm1 to 25% */
pwm2.write(0);                /* Set the Duty Cycle for pwm2 to 0% */
```

RODOS HAL (ADC):

```
class HW_HAL_ADC; /* hal_adc.h */

class HAL_ADC : public GenericIOInterface {
    HW_HAL_ADC *context;

public:
    HAL_ADC(ADC_IDX idx);

    int32_t init(ADC_CHANNEL adc_Channel);
    int32_t config(ADC_PARAMETER_TYPE type, int32_t value);

    void reset();

    int32_t read(ADC_CHANNEL channel);

    virtual bool isWriteFinished(){return false;}
    virtual bool isReadFinished(){return true;}
};
```

```
enum ADC_IDX {          // STM32F4
ADC_IDX1,               //    X
ADC_IDX2,               //    X
ADC_IDX3                //    X
};
```

```
enum ADC_PARAMETER_TYPE {
ADC_PARAMETER_RESOLUTION
};
```

```
enum ADC_CHANNEL {  // STM32F4
                    // ADC1/2/3
    ADC_CH_000,     // PA0/PA0/PA0
    ADC_CH_001,     // PA1/PA1/PA1
    ADC_CH_002,     // PA2/PA2/PA2
    ADC_CH_003,     // PA3/PA3/PA3
    ADC_CH_004,     // PA4/PA4/PF6
    ADC_CH_005,     // PA5/PA5/PF7
    ADC_CH_006,     // PA6/PA6/PF8
    ADC_CH_007,     // PA7/PA7/PF9
    ADC_CH_008,     // PB0/PB0/PF10
    ADC_CH_009,     // PB1/PB1/PF3
    ADC_CH_010,     // PC0/PC0/PC0
    ADC_CH_011,     // PC1/PC1/PC1
    ADC_CH_012,     // PC2/PC2/PC2
    ADC_CH_013,     // PC3/PC3/PC3
    ADC_CH_014,     // PC4/PC4/PF4
    ADC_CH_015,     // PC5/PC5/PF5
    ADC_CH_016,     // intern
    ADC_CH_017,     // intern
    ADC_CH_018      // intern
};
```

Example:

```
#define ADCRes              4095    // ADC full scale resolution of 12 bits = 2^12-1 = 4095
#define ADCRef              3000.0  // ADC adc reference voltage  = 3V = 3000mV
#define CurrentVoltageRatio 500     // Current-sense feedback voltage of approximately 500 mV per amp
#define Voltage              ADC_CH_001 // PA1 pin

HAL_ADC CurrentADC(ADC_IDX1);       /* declare HAL_ADC for ADC_IDX1, please refer to hal_adc.h for correct ADC mapping */
CurrentADC.config(ADC_PARAMETER_RESOLUTION, 12);

CurrentADC.init(Voltage);  /* initialization of the HAL object should be called one time only in the project*/

float ADCValue = CurrentADC.read(Voltage);
float MotorCurrent = (ADCVaule / ADCRes) * (ADCRef / CurrentVoltageRatio) * 1000;
```

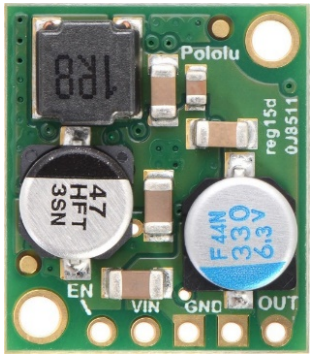
Power Source:

2x 2100mAh 2S1P 20C LiFePo4 Battery Pack:

- Capacity: 2100mAh
- Voltage: 2S1P / 2 Cell / 6.6V (7.2V fully charged)
- Discharge: 20C Constant / 40C Burst
- Weight: 103g (including wire, plug & case)
- Dimensions: 80 x 44 x 16mm
- Balance Plug: JST-XH
- Discharge Plug: JR & Futaba

• Voltage Regulator:

- **Pololu 5V, 5A Step-Down Voltage Regulator D24V50F5**
- **Minimum operating voltage: 6 V**
- **Maximum operating voltage: 38 V**
- **Continuous output current: 5 A**
- **Output voltage: 5 V (with 4% accuracy)**
- **Built-in reverse-voltage protection, short-circuit protection, a thermal shutdown feature that helps prevent damage from overheating, a soft-start feature that reduces inrush current, and an under-voltage lockout.**



Floating Satellite System (FloatSat)

Operation Modes & Requirements

Ground Station GUI & Operation Modes:

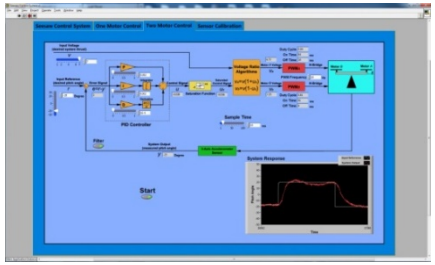
What are the requirements?

- Implementing the Telecommand and the Telemetry communication protocols for communication with the satellite
- Implementing an easy to use user interface where all the necessary information can be gathered from the satellite and displayed on the GUI using charts, numerical, string and LED indicators.

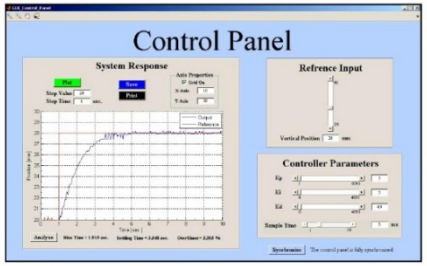
Operation Modes:

- **Standby Mode.**
- **Motor Speed Mode:** CW and CCW **rotation** with **desired motor rotation speed** — **Controller 1**
- **Velocity Mode:** CW and CCW **rotation** with **desired angular velocity** — **Controller 2**
- **Position Mode: Stabilizing** at **desired orientation** (Compass Angle) — **Controller 3**
- **Mission Mode**

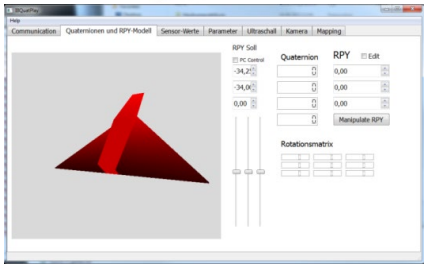
LabView, Matlab, QT or …?



Graphical Programming



M-Code



C++

Floating Satellite System (FloatSat)

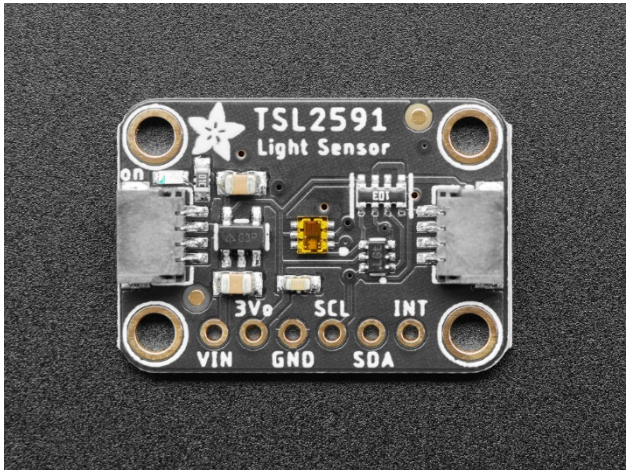
Modules & Sensors for Satellite Mission

- # Light Sensor:

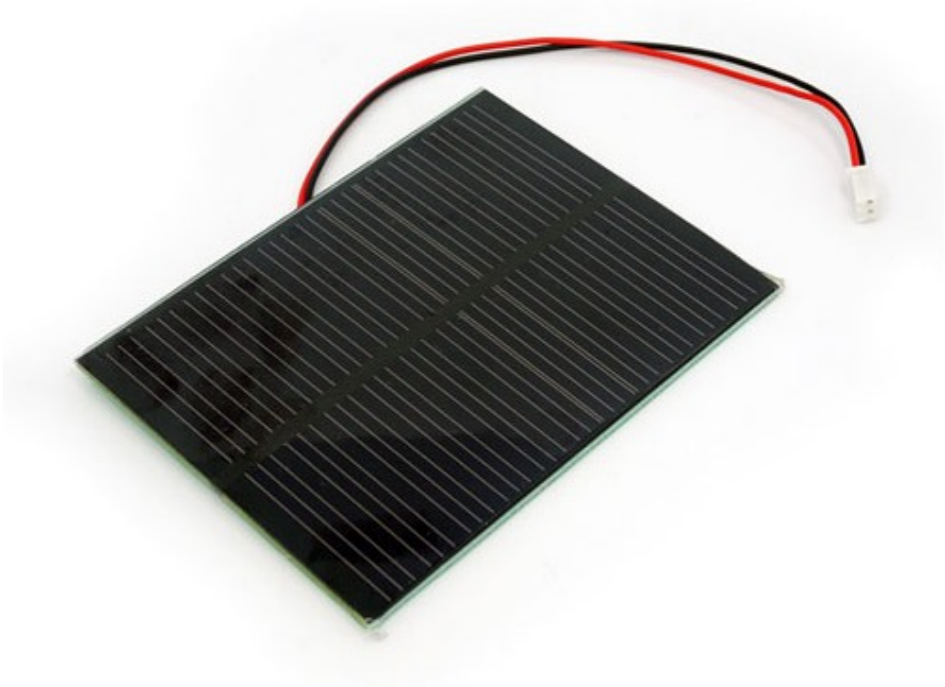
- **TSL2591** High Dynamic Range Digital Light Sensor
- Dynamic range (Lux): **188 uLux to 88,000 Lux**
- **Programmable Interrupt Function** with User-Defined Upper and Lower Threshold Settings
- Voltage range: **3.3-5V**
- **16-Bit** Digital Output with **I2C Fast-Mode** at 400 KHz

Optional use:

- Deployment of a Solar Panels.



- **2x Solar Panel**
- **PET package**
- **Typical peak power : 0.935W**
- **Voltage at peak power : 5.5V**
- **Current at peak power : 170mA**
- **Open-circuit voltage: 8.2 V**
- **Maximum load voltage: 6.4 V**
- **Dimensions: 100x80x2.5(±0.2) mm**
- **Efficiency : 15.5%**
- **Weight : 0.04 Kg**

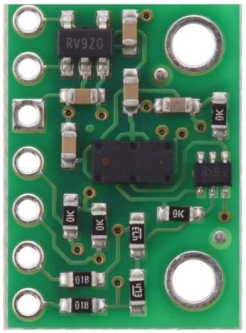


Optional use:

- **Deployment of a Solar Panels.**
- **Battery charging.**

• Laser-Ranging Sensor:

- VL53L1X Time-of-Flight Distance Sensor Carrier with Voltage Regulator
- Dimensions: 13 mm × 18 mm × 2 mm
- Operating voltage: 2.6 V to 5.5 V
- Maximum range: 400 cm with Resolution of 1 mm
- Minimum range: 4 cm
- Emitter: 940 nm invisible Class 1 – eye-safe
- Typical full field of view (FoV): 27°
- Interface: I^2C



Optional use:

- 2D or 3D mapping of an unknown terrain on the surface of a planet or asteroid.
- Space Debris Removal.

• Tasks: Actuators & Sensors

Create 4 threads in the project, each of them should toggle different LED when it is running and perform the following:

>> The first thread is the Modes thread that needs to run at 20ms time interval and with following requirements:

● Two PWM signals need to be generated on PE13 and PE11.

● A MotorSet function need to be implement to drive the reaction wheel with different speeds and in both directions CW and CCW:

```
void MotorSet(float Value)
```

>>The second thread is the Sensors thread that needs to run at 5ms time interval with following requirements:

● An ADCUpdate function need to be implement In order to be able to measure the current of the motor using using one ADC channels on PA1.

● The measured ADC values need to be filtered using low pass mean filter in order to reduce the noise in the measurements. Implement low pass mean filter with highest order possible.

● A MotorSpeedUpdate function need to be implement In order to be able to measure the speed of the motor and the direction of rotation using two GPIO Pins at PA3 and PA5.

>>The third thread is the Telemetry thread where the actual motor speed and the direction and the measured current of the motor are the telemetry data need to be sent through UART when needed.

>>The fourth thread is the Telecommand thread where the duty cycle and the motor direction of rotation should be able to update through UART with Telecommands.